

SYSTEM DESIGN · CURATOR

Curator Architecture

How curator is wired today, where the VPN boundary actually sits — including three paths that escape it — and what it takes to make the kill switch absolute.

0.2.0

one Go binary + embedded UI

Pi 5 · arm64

six containers, five services

D27

the VPN is mandatory

3 leaks found

one carries payload

01 · The claim the system rests on

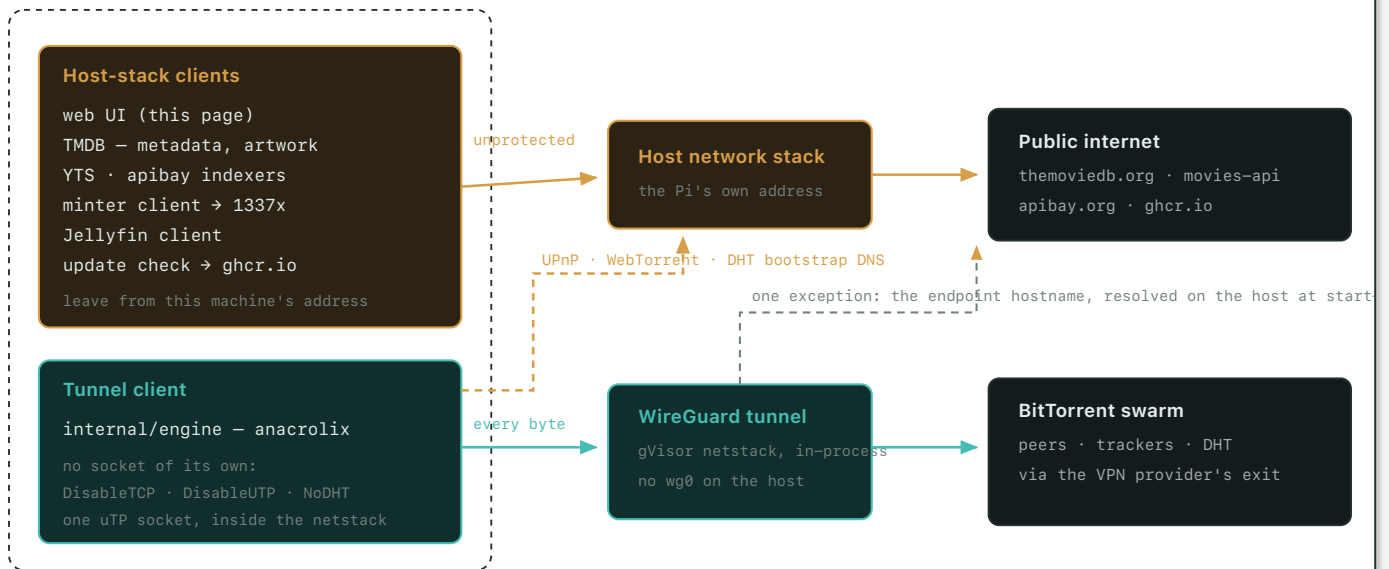
What actually crosses the tunnel

curator's guarantee is **structural, not configured**. The torrent engine is built with `DisableTCP`, `DisableUTP` and `NoDHT`, so it owns no socket and cannot make one. Every peer byte has to pass through a dialer the tunnel handed it — a dead tunnel is a failed dial, not a leak.

That is stronger than qBittorrent's "bind to `wg0`", which only tells libtorrent which address to *prefer* while leaving it able to open sockets. curator has no interface to bind to at all: the tunnel is a userspace WireGuard device on a gVisor netstack living *inside the process*, which is why it needs no `NET_ADMIN`, no privileged container and no sidecar.

The honest limit is scope. Only the engine is bound to it.

CURATOR PROCESS — ONE GO BINARY



☐ through the tunnel ☐ host stack — deliberately not tunnelled

Three paths escape the tunnel today, and they share one cause. `cfg.hermetic` hardens the test config in ways production never got — `NoDefaultPortForwarding`, `DhtStartingNodes` — so the test configuration is safer than the production one. That is exactly why none of it was caught.

THE ANSWER

Does this guarantee not one byte of download traffic?

For payload — yes, and structurally rather than by enumeration, once

`DisableWebtorrent` is set. Piece data, the metadata dictionary and webseed content can reach curator through exactly two channels: a peer connection, or an HTTP webseed. Peer connections come only from `cl.dialers` and `cl.listeners`, and with TCP and uTP disabled anacrolix builds *no sockets of its own* — `listenAll` returns `nil` before it opens anything — so both lists contain only the tunnel socket. Webseeds go through `HTTPTDialContext`, which is the tunnel dialer. WebTorrent is the third channel and the only unbound one; closing it leaves no path that is not the tunnel.

For torrent-identifying metadata — not yet. Two paths still announce that this address runs a BitTorrent client: UPnP's SSDP multicast, and the DHT bootstrap resolving eight well-known hostnames on the host's DNS. Neither moves film data. Both are closed by this plan.

The residual risk is a library upgrade, not a bug. The guarantee above is an audit of anacrolix v1.61.0's egress surface, and three paths were already missed once. That is why the plan pins the `ClientConfig` field set by reflection: a future version that adds an egress path fails the build until someone classifies it, instead of shipping quietly.

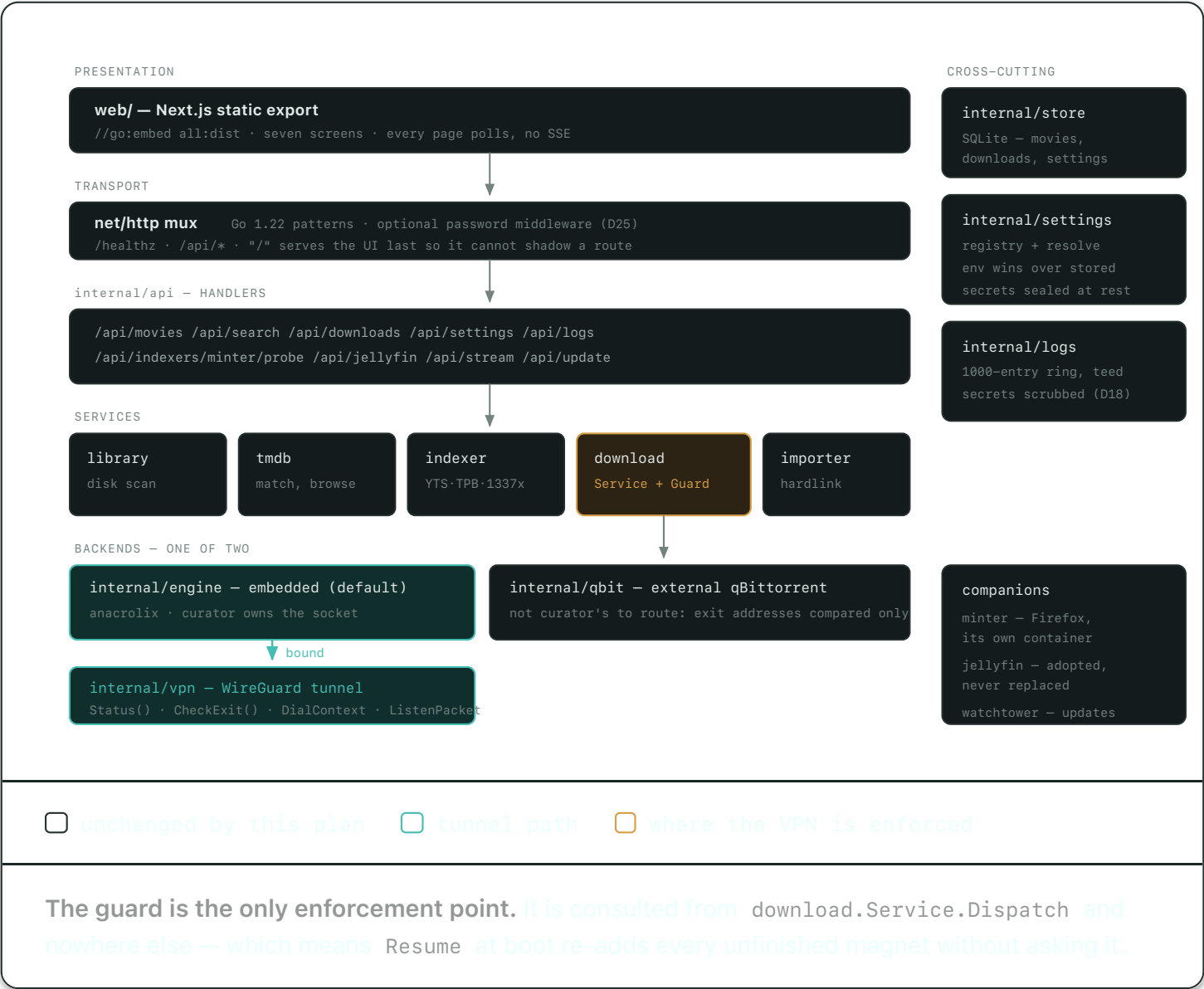
CHANNEL	CARRIES	BOUND TO THE TUNNEL?
Peer connections (uTP)	PAYLOAD	YES anacrolix creates zero sockets; only the tunnel socket is installed
Webseeds (HTTP)	PAYLOAD	YES HTTPDialContext
WebTorrent / WebRTC	PAYLOAD	NO its own websocket.Dialer — closed by this plan
Trackers (HTTP + UDP)	metadata	YES TrackerDialContext , TrackerListenPacket
DHT packets	metadata	YES server built on the tunnel socket
DHT bootstrap DNS	metadata	NO host resolver, 8 hostnames — closed by this plan
UPnP / SSDP	nothing	NO host multicast — closed by this plan

The system today

One binary, one SQLite file, one embedded UI. Every optional dependency is a nil-

tolerant field on the API server, and a nil is always a supported configuration with a sentence attached rather than a startup failure.

The VPN appears in exactly two places: as one row in the settings integrations table behind an opt-in probe button, and as a guard consulted once per dispatch. There is no screen for it.

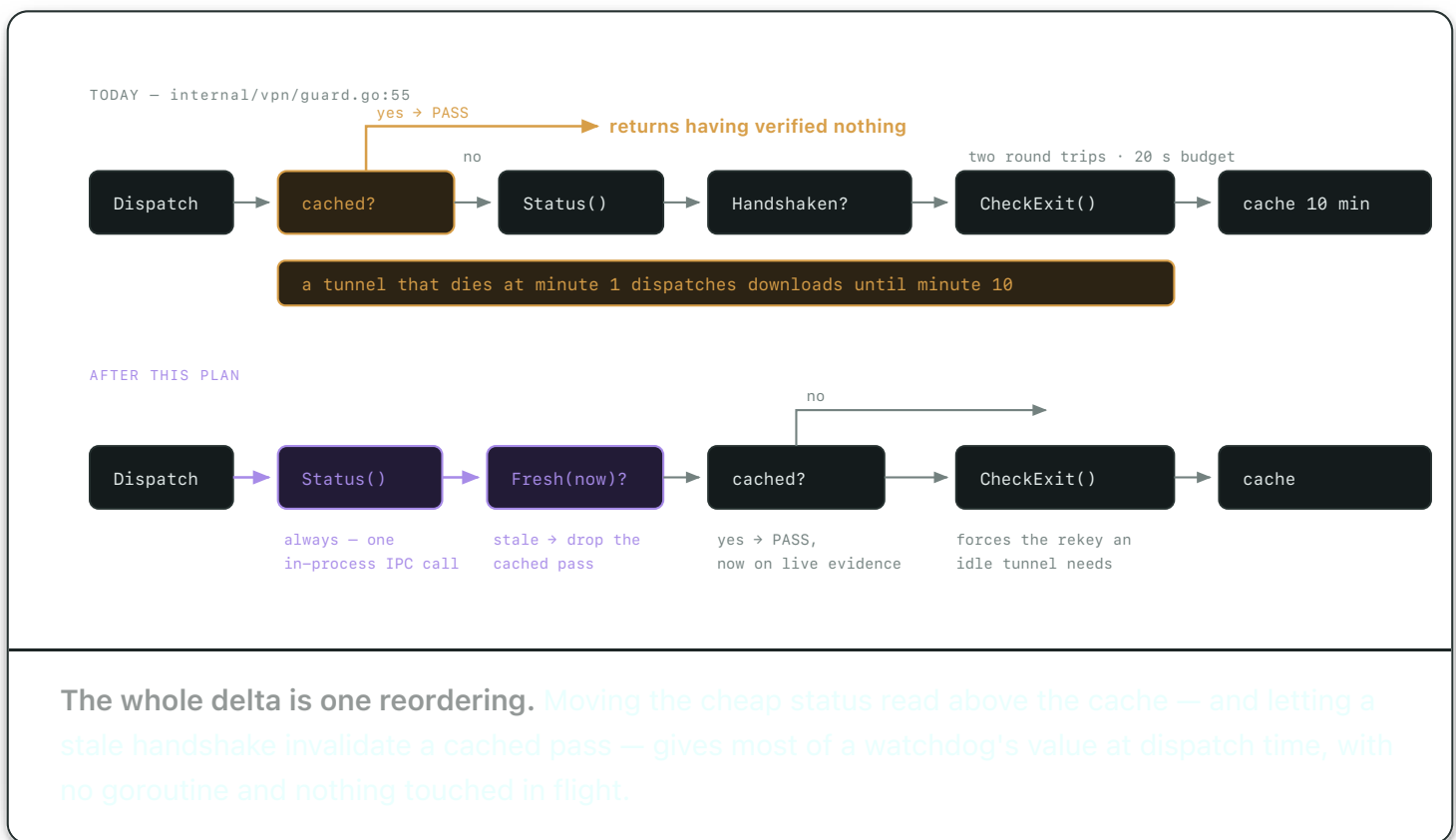


03 · The defect

A ten-minute window with no check at all

The guard caches a successful verification for ten minutes, which is sound reasoning — dispatching six films should not mean six round trips to an IP-echo service. But the cache check sits **above** the tunnel status read, so inside that window a dispatch performs no verification whatsoever, not even the free in-process one.

A tunnel that dies at minute one lets every download through until minute ten. For a system whose first principle is that the VPN is mandatory, this is the defect that matters most — and closing it costs one IPC call per dispatch and needs no background watchdog.



A SECOND CORRECTNESS PROBLEM

Status.Handshaken() is !LastHandshake.IsZero() — it means *has ever* *handshaken*, and nothing compares it to now. A tunnel whose peer vanished an hour

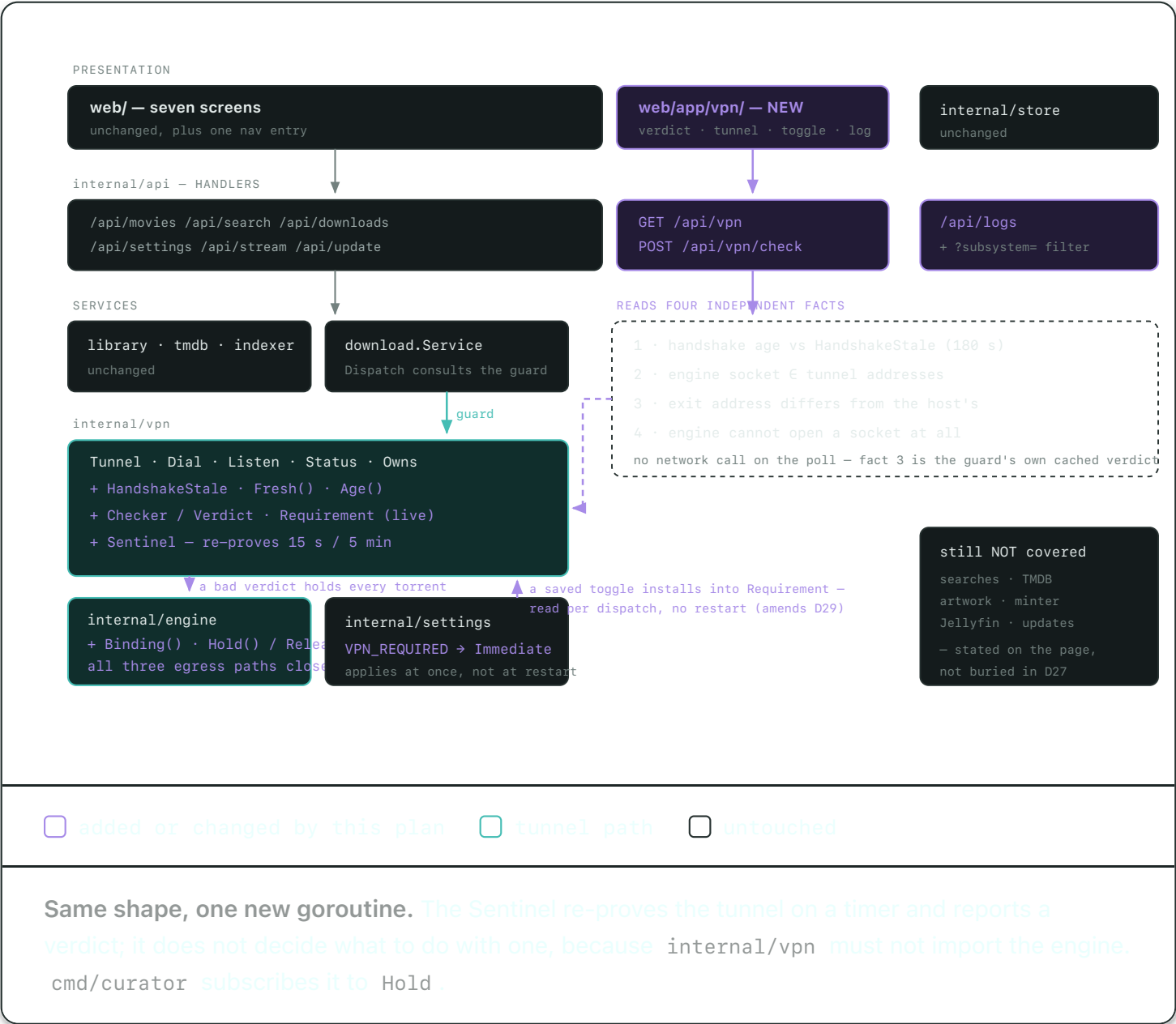
ago still reports up. A live monitor built on that reading would confidently lie, so the plan adds a real staleness notion: 180 seconds, taken from WireGuard's own `REJECT_AFTER_TIME` rather than picked as a round number.

04 · Proposed

The system after this plan

The same architecture, with one new screen, one new endpoint group, and three packages that learn to answer questions they already had the data for. Nothing is restructured; the additions are marked below.

The page proves protection from **four independently-read facts** — and the important one is that the engine's actual socket address is checked against the addresses the tunnel actually owns. That is curator's equivalent of "bound to `wg0`", except it is derived rather than configured.



05 · Delta

Changes by package

PACKAGE	CHANGE
internal/engine	FIX DisableWebtorrent (the one that carries payload),

		NoDefaultPortForwarding , and a DhtStartingNodes that resolves the bootstrap hosts <i>through the tunnel</i> — all in the <i>production</i> path, not only under <code>cfg.hermetic</code> .
internal/engine	NEW	The guard that survives an upgrade. A test reflects over <code>anacrolix.ClientConfig</code> and pins its field set: a future version that adds an egress path fails the build until someone classifies it, instead of shipping quietly the way these three did.
internal/vpn	NEW	<code>LookupHost</code> over the netstack resolver — T37 planned a resolver on the tunnel and it was never built. This is where it was needed.
internal/vpn	NEW	<code>Sentinel</code> — one goroutine: cheap status read every 15 s, full <code>CheckExit</code> every 5 min and immediately on a stale handshake. Fails closed on a status error, a stale handshake, or <code>ErrSameExit</code> .
internal/engine	NEW	<code>Hold()</code> / <code>Release()</code> — <code>DisallowDataDownload</code> + <code>DisallowDataUpload</code> + <code>SetMaxEstablishedConns(0)</code> . <code>NotDrop()</code> : a tunnel blip must not lose a half-finished download.
internal/download	FIX	<code>Resume</code> at boot now consults the guard. On a box that reboots into a broken tunnel, today it re-adds the whole library unprotected.
internal/vpn	NEW	<code>HandshakeStale</code> (180 s), <code>Status.Fresh/Age</code> , <code>Since</code> , <code>Keepalive</code> , <code>Tunnel.Owns</code> . <code>Handshaken()</code> is left untouched — the guard still needs it.
internal/vpn	CHANGED	The cheap <code>Status()</code> read moves above the cache; a stale handshake drops a cached pass. <code>Checker / Verdict</code> retain the exit address in memory — never logged. <code>Requirement</code> + <code>Enforced</code> replace construction-time <code>Advisory</code> .

internal/engine	NEW	Binding() — the socket the engine actually holds, read off the live client rather than the config it was built from.
internal/api	NEW	vpn.go — GET /api/vpn (cheap, always 200) and POST /api/vpn/check (two round trips, rate-limited). Auth.Enforcing() gates the exit address.
internal/api	CHANGED	?subsystem= filter on /api/logs , applied after the cursor is computed so an idle poll stays empty.
internal/settings	CHANGED	VPN_REQUIRED gains Immediate: true . Credentials (VPN_CONFIG) stay restart-required under D29.
cmd/curator	CHANGED	A composed live []api.Access applier, a vpnEnforcement adapter resolving through config.Load , the settingsStates fix, and log.With("subsystem", "vpn") .
web/	NEW	app/vpn/ plus four components; logline.tsx extracted from the logs page rather than duplicated.

A TRAP THAT WOULD HAVE SHIPPED QUIETLY

Making VPN_REQUIRED immediate is not a one-line change. settingsStates reports an immediate setting's live value as the raw stored string — and VPN_REQUIRED defaults to true with no stored row on a fresh install. As written, the settings form would draw **the kill switch off while enforcement is on**.

The fix is to name the real exception: the two Access settings have no *config.Config field, which is what makes reading the stored string correct for them. Everything else must resolve through config.Load and report the default it lands on.

06 · Limits

What this still will not do

Stated here, and stated on the page itself, because a monitor that overclaims is worse than no monitor. The two entries that used to sit here — nothing watching while the page is closed, and `Resume` not asking the guard — are now in scope rather than accepted.

Only peer traffic is tunnelled. Searches, TMDB, artwork, Jellyfin and the update check leave from this machine's address. Routing them is wiring rather than a rewrite — every one takes an `*http.Client` — but it is its own task.

minter cannot be fixed from here. It browses 1337x with a real Firefox from its own container, which curator does not route at all. That needs minter to have its own VPN.

The payload guarantee is structural; the rest is an audit. Piece data cannot leave the tunnel because no other socket exists. Everything else rests on having enumerated anacrolix v1.61.0's egress surface correctly — which is why the reflection test exists, and why an upgrade is a review rather than a bump.

Worst-case exposure is bounded, not zero. A dead peer is caught within about 15 seconds, a silently-degraded exit within about 5 minutes. Making either continuous means checking on every piece, which costs more than it buys.

curator 0.2.0 · architecture as of 2026-08-19 · byte paths audited against
anacrolix v1.61.0 · proposed work is T82–T84 and D47, not yet built